

ACCIDENT PREVENTION SYSTEM AT BLIND CURVES

A Mini Project Work

Submitted in partial fulfilment of the requirements for the award of
the degree of

**BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING**

By

AVUTHU ROOPESH REDDY - 24EG504E01

Under the guidance of

Dr. RAJESH THUMMA

Associate Professor Department of ECE



Department of Electronics and Communication Engineering

ANURAG UNIVERSITY SCHOOL OF ENGINEERING

Venkatapur(V), Ghatkesar(M), Medchal-Malkajgiri Dist-500088

2026-2027

ANURAG UNIVERSITY
SCHOOL OF ENGINEERING
Venkatapur(V), Ghatkesar(M), Medchal-Malkajgiri Dist-500088



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

CERTIFICATE

This is to certify that the project report entitled “**ACCIDENT PREVENTION SYSTEM AT BLIND CURVES**” being submitted by

AVUTHU ROOPESH REDDY- 24EG504E01

in partial fulfillment for the award of the Degree of Bachelor of Technology in Electronics & Communication Engineering to the Anurag University, Hyderabad is a record of bonafide work carried out under my guidance and supervision. The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Dr. Rajesh Thumma
Associate Professor

Dr. Harikrishna Kamatham,
Professor, HOD
ECE & ECM

External Examiner

ACKNOWLEDGEMENT

This project is an acknowledgement to the inspiration, drive and technical assistance contributed by many individuals. This project would have never seen light of this day without the help and guidance I have received. I would like to express my gratitude to all the people behind the screen who helped me to transform an idea into a real application.

It's my privilege and pleasure to express my profound sense of gratitude to **Dr. Rajesh Thumma, Associate Professor**, Department of ECE for his guidance throughout this dissertation work. I express my sincere gratitude to **Dr. Harikrishna Kamatham, Professor, Head of the Department**, ECE & ECM for his precious suggestions for the successful completion of this project. He is also a great source of inspiration to my work.

I would like to express my deep sense of gratitude to **Dr. V. Vijaya Kumar, Dean School of Engineering**, Anurag University for his tremendous support, encouragement and inspiration. Lastly, I thank almighty, my parents, friends for their constant encouragement without which this assignment would not be possible. I would like to thank all the other staff members, both teaching and non- teaching, which have extended their timely help and eased my work.

BY

AVUTHU ROOPESH REDDY - 24EG504E01

DECLARATION

I hereby declare that the result embodied in this project report entitled “**ACCIDENT PREVENTION SYSTEM AT BLIND CURVES**” is carried out by me during the year 2026-2027 for the partial fulfilment of the award of **Bachelor of Technology in Electronics and Communication Engineering**, from ANURAG UNIVERSITY. I have not submitted this project report to any other Universities / Institute for the award of any degree.

By

AVUTHU ROOPESH REDDY - 24EG504E01

ABSTRACT

Road accidents at blind curves are a major concern, particularly in hilly and mountainous regions where drivers have limited visibility of approaching vehicles. Conventional safety measures such as warning signs, convex mirrors, and speed breakers are passive in nature and often fail to provide real-time information about oncoming traffic. To address this issue, this project presents an Accident Prevention System at Blind Curves that provides early warnings to drivers using wireless communication and intelligent decision-making.

The proposed system consists of two ESP32-based sensing units installed on either side of the blind curve and a central control unit comprising an ESP8266 and an Arduino Nano. Each sensing unit uses ultrasonic sensors to detect approaching vehicles, estimate vehicle height, calculate speed, and determine the estimated time required to reach the curve. The collected information is transmitted through LoRa communication, which enables reliable long-range wireless data transfer with low power consumption.

The central controller processes the received data and implements a priority-based algorithm that considers vehicle direction, speed, type, and estimated arrival time. Based on these parameters, the system generates appropriate warning messages and activates LEDs, an LCD display, and a buzzer to alert drivers about approaching traffic. Giving priority to uphill vehicles further improves safety and traffic management on inclined roads.

The developed prototype demonstrates reliable vehicle detection, efficient wireless communication, and timely warning generation under different operating conditions. The proposed system is cost-effective, easy to install, and suitable for remote locations where conventional traffic monitoring infrastructure is unavailable. By providing real-time alerts before vehicles enter the blind curve, the system enhances driver awareness and significantly reduces the possibility of head-on collisions. The modular design also allows future enhancements such as solar-powered operation, GPS integration, and cloud-based traffic monitoring, making it a practical solution for improving road safety.

TABLE OF CONTENTS

CHAPTER NO	DESCRIPTION	PAGE NO
	CERTIFICATE	ii
	ACKNOWLEDGEMENT	iii
	DECLARATION	iv
	ABSTRACT	v
CHAPTER 1 :	INTRODUCTION	1 - 5
	1.1 INTRODUCTION	1
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVE	4
CHAPTER 2:	LITERATURE SURVEY	5 – 8
CHAPTER 3:	METHODOLOGY	9 – 13
	3.1 SYSTEM OVERVIEW	9
	3.2 BLOCK DIAGRAM	10
	3.3 FLOWCHART	10
	3.4 SYSTEM ARCHITECTURE	11
CHAPTER 4:	HARDWARE AND SOFTWARE REQUIREMENTS	14- 23
	4.1 HARDWARE DESCRIPTION	14
	4.2 SOFTWARE DESCRIPTION	22
CHAPTER 5:	RESULTS AND DISCUSSION	24 – 28
CHAPTER 6:	CONCLUSION AND FUTURE SCOPE	29 – 30
	6.1 CONCLUSION	29
	6.2 FUTURE SCOPE	30
	REFERENCES	30 - 31
	APPENDIX	32.- 68

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Blind Curve	1
3.1	Block Diagram	10
3.2	Flow Chart of Sender	10
3.3	Flow Chart of Receiver at Blind Curve	11
4.1	ESP32	15
4.2	NodeMCU	16
4.3	Arduino Nano	17
4.4	Ultrasonic Sensor	18
4.5	LoRa Module	19
4.6	16x2 LCD Display	20
4.7	LED's	21
4.8	Buzzer	21
5.1	Prototype	24
5.2	Upper Node	25
5.3	Lower Node	26
5.4	Blind Curve Node	27

ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
DC	Direct Current
ESP	Espressif Systems Processor
ETA	Estimated Time of Arrival
GPIO	General Purpose Input/Output
HC-SR04	Ultrasonic Distance Sensor Module
I ² C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LoRa	Long Range
MCU	Microcontroller Unit
MHz	Megahertz
RF	Radio Frequency
SPI	Serial Peripheral Interface
UART	Universal Asynchronous Receiver/Transmitter
VCC	Voltage at Common Collector (Power Supply)
Wi-Fi	Wireless Fidelity

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Road transportation is one of the most important modes of transportation in the modern world, providing an efficient means for the movement of people and goods. As the number of vehicles on roads continues to increase, ensuring road safety has become one of the primary concerns for transportation authorities. Although significant improvements have been made in vehicle safety systems and road infrastructure, accidents continue to occur frequently due to environmental conditions, human errors, and inadequate traffic management systems. Among the various accident-prone locations, blind curves are considered one of the most dangerous because they restrict the visibility of approaching vehicles, making it difficult for drivers to react in time.

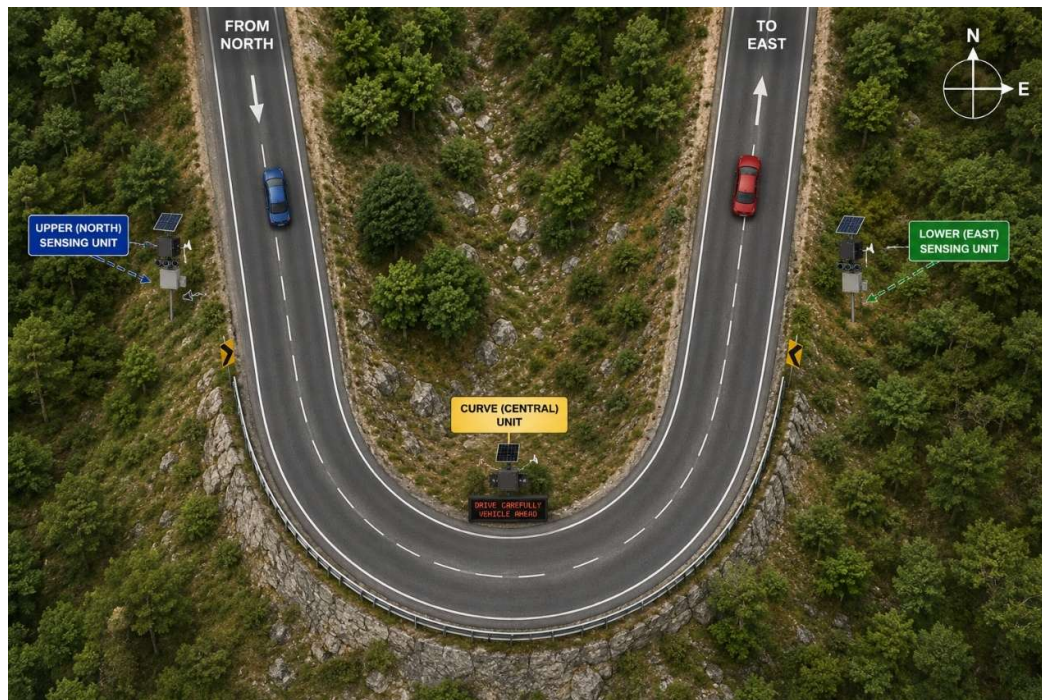


Fig 1.1 Blind Curve

Blind curves are commonly found in hilly regions, mountainous roads, forest routes, and sharp bends where the driver's line of sight is blocked by natural terrain or road geometry. At such locations, drivers approaching from opposite directions are unable to see each other until they are very close to the turning point. This limited

visibility significantly increases the probability of head-on collisions, especially when vehicles travel at high speeds or when heavy vehicles occupy a larger portion of the roadway. During adverse weather conditions such as rain, fog, or low-light environments, the risk becomes even greater because the driver's visibility is further reduced.

Traditional safety measures such as traffic signboards, convex mirrors, speed breakers, reflective markers, and road markings provide only passive assistance to drivers. These methods depend entirely on the driver's attentiveness and judgment and cannot provide real-time information regarding approaching vehicles. In many remote or hilly regions, installing sophisticated traffic monitoring infrastructure is also difficult because of geographical constraints and the lack of communication facilities. Therefore, there is a growing need for an intelligent and autonomous warning system capable of detecting approaching vehicles and providing timely alerts before vehicles enter the blind curve.

The proposed Accident Prevention and Detection System at Blind Curves addresses this problem by combining embedded systems, wireless communication, and sensor technology to create an efficient real-time vehicle warning system. The system consists of two ESP32-based sensing units installed on opposite sides of a blind curve. Each sensing unit continuously monitors approaching vehicles using multiple ultrasonic sensors. The sensing units not only detect the presence of a vehicle but also estimate its height for basic classification, calculate its speed, and estimate the time required for the vehicle to reach the blind curve. This information is transmitted wirelessly through LoRa (Long Range) communication technology, enabling reliable communication over long distances with very low power consumption.

A central processing unit consisting of an ESP8266 and an Arduino Nano receives the information transmitted by both sensing units. The controller analyzes the received data, compares the direction, speed, vehicle type, and estimated arrival time of vehicles approaching from both sides, and executes a priority-based decision algorithm. Since vehicles moving uphill generally require greater effort to restart after stopping, the system provides higher priority to uphill traffic whenever conflicting situations occur. Based on the calculated priority, the controller activates warning

devices including LED indicators, LCD displays, and a buzzer to inform drivers whether it is safe to proceed or whether they should wait for the approaching vehicle to pass.

Unlike conventional blind curve warning systems that only indicate the presence of a vehicle, the proposed system provides additional information such as vehicle type, estimated speed, and arrival time. The use of LoRa communication eliminates the need for wired connectivity between both sides of the curve, making the system suitable for deployment in remote locations where communication infrastructure is limited. The modular architecture also simplifies installation, maintenance, and future expansion.

The system has been developed using cost-effective hardware components including ESP32 development boards, an ESP8266 controller, an Arduino Nano, HC-SR04 ultrasonic sensors, LoRa SX1278 wireless modules, LCD displays, LEDs, and a buzzer. These components collectively provide an economical yet efficient solution for improving road safety at accident-prone blind curves.

The primary objective of this project is to reduce the probability of head-on collisions by providing drivers with accurate and timely information about approaching vehicles. By combining real-time sensing, wireless communication, and intelligent decision-making, the proposed system enhances driver awareness, improves traffic management at blind curves, and contributes to safer transportation in hilly and mountainous regions.

1.2 PROBLEM STATEMENT

Blind curves are among the most accident-prone sections of road networks, particularly in hilly, mountainous, and forest regions where the driver's line of sight is obstructed by road geometry or natural terrain. At these locations, drivers approaching from opposite directions are unable to see each other until they are very close to the curve, leaving very little time to react. This limited visibility significantly increases the possibility of head-on collisions, especially when vehicles are travelling at high speeds or when large vehicles occupy a greater portion of the roadway.

Conventional safety measures such as warning signboards, convex mirrors, road markings, and speed breakers are passive in nature and rely entirely on the driver's awareness and judgment. These methods cannot provide real-time information regarding approaching vehicles and become less effective during adverse weather conditions such as rain, fog, or at night. As a result, drivers often make decisions without knowing whether another vehicle is approaching from the opposite side of the curve.

Another major challenge is the absence of an intelligent communication system between both sides of the blind curve. Without continuous monitoring and information exchange, drivers cannot accurately estimate the position, speed, or arrival time of approaching vehicles. This problem becomes more critical on narrow roads where heavy vehicles such as buses and trucks require additional road space and longer stopping distances.

To overcome these limitations, there is a need for a smart, reliable, and cost-effective accident prevention system capable of continuously detecting approaching vehicles, estimating their speed and arrival time, and communicating this information in real time. The system should analyze the received data and provide timely visual and audible warnings to drivers before they enter the blind curve. Such a system would improve driver awareness, support safer decision-making, and significantly reduce the risk of head-on collisions, thereby enhancing road safety in blind curve environments.

1.3 OBJECTIVE

The primary objective of this project is to design and develop an intelligent Accident Prevention and Detection System at Blind Curves that improves road safety by providing real-time information about approaching vehicles. The system aims to reduce the possibility of head-on collisions through continuous vehicle monitoring, wireless communication, and timely warning generation.

The specific objectives of the proposed system are as follows:

- To detect and classify approaching vehicles on both sides of a blind curve by

estimating their speed, height, and arrival time using ultrasonic sensors.

- To process and communicate vehicle information using LoRa, ESP8266, and Arduino Nano, and determine traffic priority through a decision-making algorithm.
- To provide timely visual and audible warnings to drivers through a low-cost, energy-efficient, and scalable system suitable for remote hilly and mountainous roads.

CHAPTER 2

LITERATURE SURVEY

Several researchers have proposed different techniques to reduce accidents occurring at blind curves and sharp bends by utilizing embedded systems, wireless communication, and intelligent sensing technologies. This chapter presents a review of some important research works related to accident prevention systems, their methodologies, advantages, limitations, and their contribution towards the development of the proposed system.

LITERATURE REVIEW

- “U-Bend Accident Prevention Through LoRa Communication” by K. V. Mutya and S. Rudra proposed a smart accident prevention system for blind curves using LoRa wireless communication and embedded controllers. The system detects approaching vehicles from opposite directions and exchanges information through LoRa modules to generate warning signals before vehicles enter the blind curve. The proposed method improves communication over long distances while consuming very low power, making it suitable for hilly and remote areas. The major advantage of this system is its reliable long-range wireless communication and simple implementation. However, the system mainly focuses on vehicle detection and warning generation and does not estimate vehicle speed, calculate the estimated time of arrival, classify vehicles, or implement an intelligent priority-based decision mechanism.

- “Embedded System for Blind Curve Accident Prevention” by P. Sharma and A. Verma proposed an embedded system using ultrasonic sensors and a microcontroller to detect vehicles approaching blind curves. The system activates LEDs and a buzzer to alert drivers whenever a vehicle is detected from the opposite direction. The proposed system is simple, economical, and easy to install in accident-prone areas. The major advantage of this system is its low implementation cost and straightforward operation. However, it does not provide wireless communication, vehicle speed estimation, vehicle classification, arrival time prediction, or intelligent traffic management, thereby

limiting its effectiveness in complex traffic conditions.

- “LoRaWAN-Based Vehicle-to-Vehicle Communication System for Collision Prevention” by Z. Zhang, Y. Wang, and X. Liu proposed a vehicle communication system based on LoRaWAN technology to improve road safety by exchanging information between moving vehicles. The system utilizes LoRa communication to achieve reliable long-distance data transmission with low power consumption, making it suitable for intelligent transportation applications. The major advantage of this system is its stable long-range communication and high reliability. However, the proposed work mainly concentrates on communication performance and does not include vehicle detection using sensors, speed calculation, estimated arrival time prediction, or driver warning mechanisms required for blind curve accident prevention.
- “IoT-Based Smart Road Safety System Using Embedded Technology” by S. Kumar and R. Singh proposed an Internet of Things (IoT)-based road safety system that continuously monitors traffic conditions using embedded controllers and sensors. The collected information is transmitted to a centralized monitoring platform for traffic management and accident prevention. The major advantage of this system is its ability to perform continuous monitoring and centralized data analysis. However, the system depends on internet connectivity and cloud infrastructure, resulting in higher implementation cost and making it less suitable for remote hilly regions where network availability is limited.
- “Performance Assessment of LoRaWAN for Vehicular Communication” by M. F. Ahmed, A. Khan, and S. Rahman presented a detailed evaluation of LoRaWAN technology for vehicular communication applications. The study analyzed communication range, packet delivery ratio, transmission reliability, and network performance under different operating conditions. The results demonstrated that LoRa provides reliable long-range communication while maintaining low power consumption. The major advantage of this work is its comprehensive evaluation of LoRa communication performance for intelligent transportation systems. However, the research focuses only on communication performance and does not implement vehicle detection, speed estimation, vehicle classification, estimated time of arrival calculation, or priority.

RESEARCH GAP

The literature survey reveals that most of the existing blind curve accident prevention systems focus primarily on detecting approaching vehicles and providing basic warning indications. Although several researchers have utilized ultrasonic sensors, embedded systems, and LoRa communication to improve road safety, many existing solutions lack advanced features such as vehicle speed calculation, estimated time of arrival (ETA) prediction, vehicle classification, and intelligent decision-making. In addition, several IoT-based systems depend on internet connectivity or cloud infrastructure, making them unsuitable for remote hilly regions where communication facilities are limited.

To overcome these limitations, the proposed Accident Prevention System at Blind Curves integrates ESP32-based sensing units with LoRa wireless communication. The system not only detects approaching vehicles but also estimates vehicle height, calculates vehicle speed, predicts the estimated time of arrival, and applies a priority-based decision algorithm. A central controller consisting of an ESP8266 and an Arduino Nano processes the received information and generates appropriate visual and audible warnings through LCD displays, LEDs, and a buzzer. This approach provides a reliable, low-cost, and efficient solution for improving road safety at blind curves, particularly in remote and mountainous areas.

SUMMARY

This chapter presented a review of five research papers related to blind curve accident prevention, embedded systems, ultrasonic sensing, LoRa communication, and intelligent transportation systems. The strengths and limitations of the existing approaches were analyzed to understand their effectiveness in improving road safety. The literature survey highlighted several research gaps, including the absence of speed estimation, arrival time prediction, vehicle classification, and intelligent priority-based traffic management in many existing systems. Based on these observations, the proposed system was developed to overcome these limitations by integrating real-time vehicle detection, speed calculation, LoRa communication, and an intelligent decision-making mechanism, thereby providing a more comprehensive and reliable solution for accident prevention at blind curves.

CHAPTER 3

METHODOLOGY

3.1 SYSTEM OVERVIEW

The proposed system consists of three interconnected modules that work together to monitor approaching vehicles and generate appropriate warning signals. These modules are the Upper Sensing Unit, Lower Sensing Unit, and the Central Control Unit.

The Upper Sensing Unit is installed on one side of the blind curve and continuously monitors vehicles approaching from the uphill direction. It consists of an ESP32 microcontroller, three ultrasonic sensors, and a LoRa communication module. The ultrasonic sensors detect the approaching vehicle, estimate its height, calculate its speed, and determine the estimated time required to reach the blind curve. After processing these parameters, the ESP32 transmits the information to the central controller through the LoRa module.

Similarly, the Lower Sensing Unit is installed on the opposite side of the blind curve. It performs the same functions for vehicles approaching from the downhill direction. The sensing and communication process remains identical, ensuring that information from both sides of the curve is continuously available for processing.

The Central Control Unit consists of an ESP8266, an Arduino Nano, two LCD displays, LEDs, and a buzzer. The ESP8266 receives data transmitted from both sensing units through LoRa communication and executes the decision-making algorithm. Based on vehicle direction, speed, estimated arrival time, and vehicle type, the controller determines the traffic condition at the blind curve. The Arduino Nano receives commands from the ESP8266 through I²C communication and controls the LED indicators, LCD displays, and buzzer to provide clear visual and audible warnings to drivers.

The complete system operates continuously and automatically without requiring human intervention. By exchanging real-time vehicle information between both sides of the blind curve and intelligently processing the collected data, the proposed system

provides drivers with sufficient warning before entering the dangerous section of the road, thereby significantly reducing the possibility of accidents.

3.2 BLOCK DIAGRAM

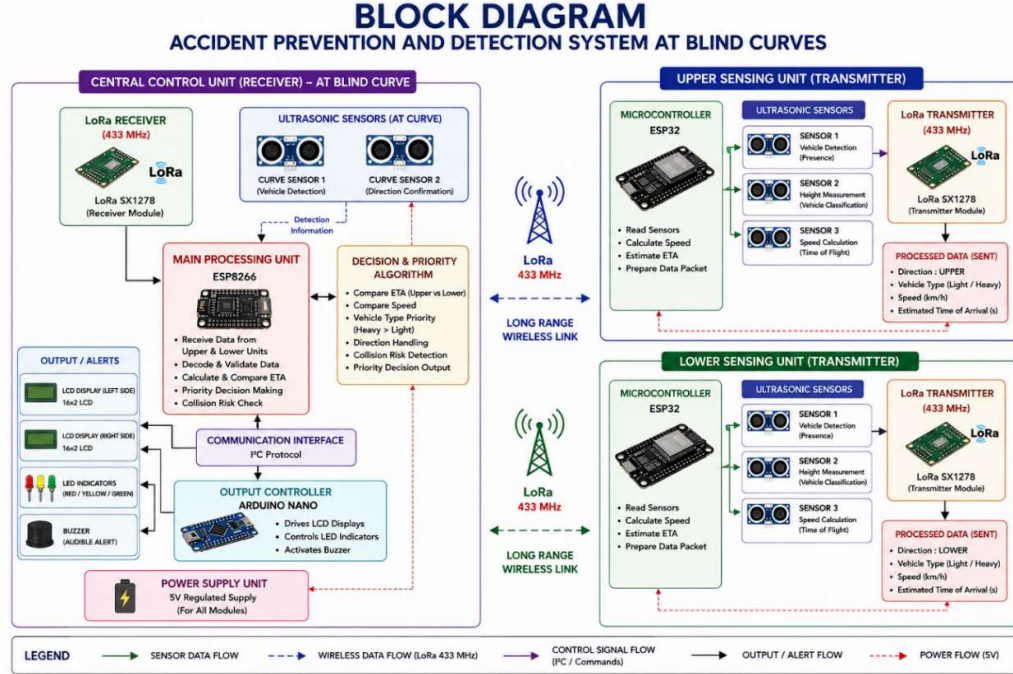


Fig 3.1 Block Diagram

3.3 FLOWCHART

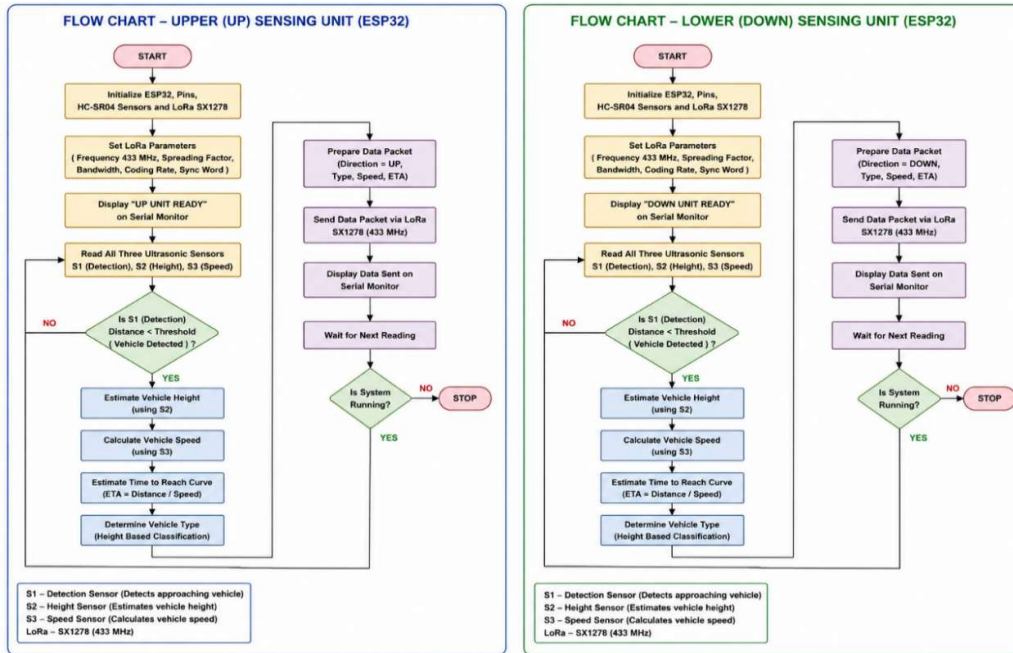


Fig 3.2 Flow Chart of Sender

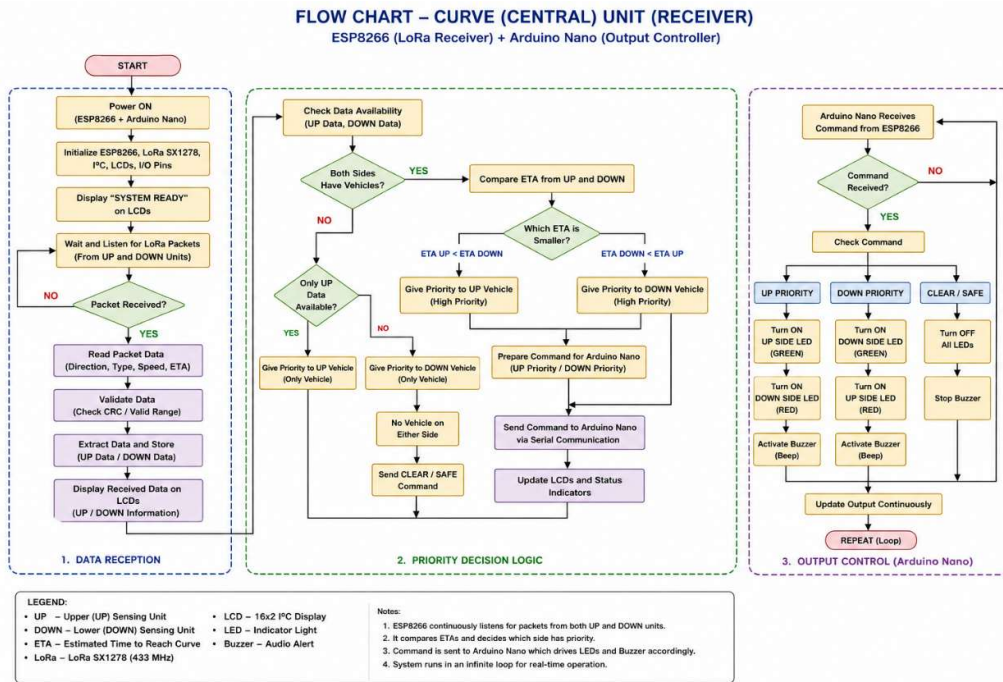


Fig 3.3 Flow Chart of Receiver at Blind Curve

3.4 SYSTEM ARCHITECTURE

The proposed Accident Prevention System at Blind Curves follows a distributed architecture consisting of two sensing units and one central control unit. The architecture is designed to continuously monitor vehicle movement on both sides of a blind curve, exchange information through long-range wireless communication, and generate appropriate warning signals for approaching drivers. The complete system integrates ESP32 microcontrollers, an ESP8266 controller, an Arduino Nano, LoRa communication modules, ultrasonic sensors, LCD displays, LEDs, and a buzzer to provide an efficient and reliable accident prevention solution.

The system consists of three major modules, namely the Upper Sensing Unit, Lower Sensing Unit, and the Central Control Unit. Each module performs a specific task while communicating with the other modules to ensure smooth and reliable operation.

Upper Sensing Unit

The Upper Sensing Unit is installed to monitor vehicles approaching from the upper road. This unit is built around an ESP32 DEV MODULE connected to three HC-SR04 ultrasonic sensors and a LoRa SX1278 transmitter module.

The first ultrasonic sensor detects the presence of an approaching vehicle. Once a vehicle is detected, the second ultrasonic sensor is used to estimate the vehicle height, enabling basic classification into light or heavy vehicles. The third ultrasonic sensor measures the travel time of the vehicle over a known distance, allowing the ESP32 to calculate the vehicle speed. Using the measured speed and the predefined distance between the sensing unit and the blind curve, the ESP32 estimates the vehicle's arrival time at the curve. The processed information, including vehicle direction, vehicle type, speed, and estimated arrival time, is transmitted to the central control unit through the LoRa module operating at 433 MHz.

Lower Sensing Unit

The Lower Sensing Unit performs functions identical to those of the Upper Sensing Unit but monitors vehicles approaching from the opposite direction. It also consists of an ESP32 development board, three ultrasonic sensors, and a LoRa transmitter module.

The ESP32 continuously detects approaching vehicles, estimates vehicle height, calculates speed, and predicts the estimated time required to reach the blind curve. The processed data is then transmitted wirelessly to the central controller using LoRa communication. Since both sensing units operate independently, the system can simultaneously monitor vehicles approaching from both directions.

Central Control Unit

The Central Control Unit is installed near the blind curve and serves as the decision-making section of the system. It consists of an ESP8266 microcontroller, an Arduino Nano, a LoRa receiver module, two LCD displays, LED indicators, a buzzer, and two ultrasonic sensors placed at the curve.

The ESP8266 receives vehicle information transmitted by both sensing units through the LoRa receiver. It validates the received data, updates the vehicle information, and compares the direction, speed, vehicle type, and estimated arrival time of vehicles approaching from both sides. Based on these parameters, it executes the priority decision algorithm to determine the appropriate traffic condition and generates suitable warning commands.

The Arduino Nano functions as the output controller. It receives commands from the ESP8266 through the I²C communication interface and controls the LCD displays, LEDs, and buzzer accordingly. Two ultrasonic sensors installed at the blind curve monitor the actual passage of vehicles. These sensors confirm when a vehicle has successfully crossed the curve, allowing the controller to update the traffic status and modify the warning indications for the next approaching vehicle.

Communication Architecture

Communication between the sensing units and the central controller is established using LoRa SX1278 modules operating at 433 MHz. LoRa technology provides reliable long-range communication with low power consumption, making it suitable for deployment in remote hilly regions where conventional communication infrastructure may not be available.

The ESP8266 and Arduino Nano communicate using the I²C protocol, ensuring fast and reliable exchange of control commands. This separation of processing and output control reduces the computational load on individual controllers.

Working Architecture

During operation, the upper and lower sensing units continuously monitor approaching vehicles and transmit processed vehicle information to the central controller. The ESP8266 analyzes the received information and determines whether vehicles can safely proceed or whether one direction should wait. Based on the decision, the Arduino Nano activates the appropriate LED indicators, LCD messages, and buzzer to guide drivers safely through the blind curve. After the vehicle passes the curve, the curve-side ultrasonic sensors detect its movement and update the system status, enabling continuous monitoring and real-time traffic management.

The modular architecture of the proposed system ensures reliable operation, simplifies maintenance, and allows future expansion with additional sensing units or communication modules. By integrating embedded processing, ultrasonic sensing, and long-range wireless communication, the proposed architecture provides an efficient, scalable, and cost-effective solution for improving road safety at blind curves.

CHAPTER 4

HARDWARE AND SOFTWARE REQUIREMENTS

The proposed Accident Prevention and Detection System at Blind Curves is developed using embedded hardware components and software tools that enable real-time vehicle detection, wireless communication, data processing, and warning generation. The hardware components are selected based on reliability, low power consumption, ease of programming, and cost-effectiveness. The software tools provide an efficient environment for developing, programming, and debugging the embedded system.

4.1 HARDWARE DESCRIPTION:

The hardware components used in the proposed system are listed below.

4.1.1 ESP32 Development Board

The ESP32 Development Board serves as the main controller for both the upper and lower sensing units. It interfaces with the ultrasonic sensors, processes sensor readings, calculates vehicle speed and estimated time of arrival (ETA), classifies vehicles based on height, and transmits the processed information to the central controller using LoRa communication.

Specifications:

- 32-bit Dual-Core Xtensa Processor
- Operating Voltage: 3.3 V
- Clock Frequency: Up to 240 MHz
- Built-in Wi-Fi and Bluetooth
- Multiple GPIO Pins

Specifications:

- 32-bit Tensilica Processor
- Operating Voltage: 3.3 V
- Clock Frequency: 80 MHz
- Built-in Wi-Fi
- UART, SPI and I²C Support

Role in the Project:

- LoRa data reception
- Vehicle data processing
- Priority decision making
- Communication with Arduino Nano

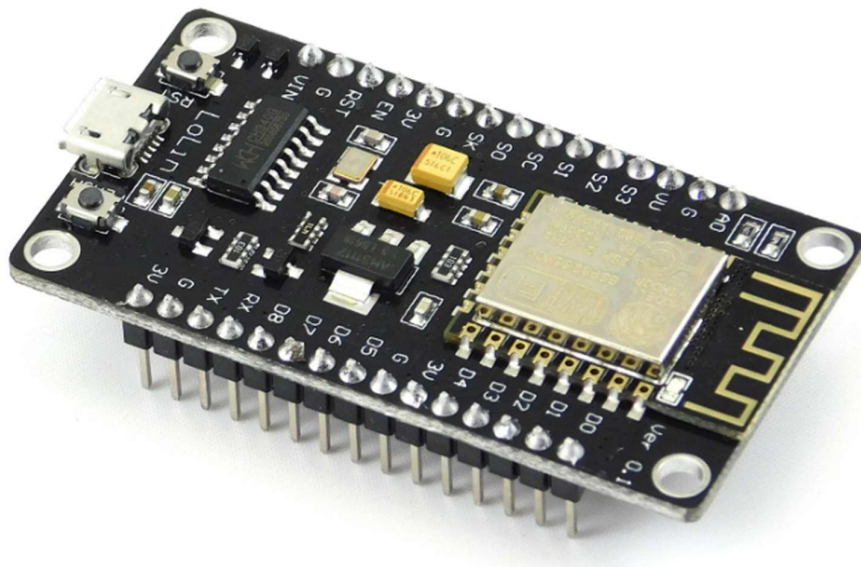


Fig 4.2 NodeMCU

4.1.3 Arduino Nano

The Arduino Nano functions as the output controller. It receives commands from the ESP8266 through the I²C communication interface and controls the LCD displays, LED indicators, buzzer, and the ultrasonic sensors positioned at the blind curve.

Specifications:

- ATmega328P Microcontroller
- Operating Voltage: 5 V
- Clock Frequency: 16 MHz
- 14 Digital I/O Pins & 8 Analog Input Pins

Role in the Project:

- LCD control
- LED control
- Buzzer control
- Curve-side ultrasonic monitoring

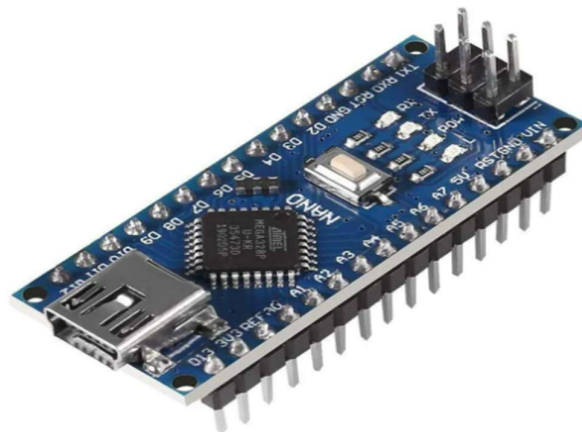


Fig 4.3 Arduino Nano

4.1.4 HC-SR04 Ultrasonic Sensor

The HC-SR04 ultrasonic sensor measures the distance between the sensor and an approaching vehicle using ultrasonic sound waves.

Specifications:

- Operating Voltage: 5 V
- Measuring Range: 2 cm to 400 cm
- Frequency: 40 kHz
- Accuracy: ± 3 mm

Role in the Project:

- Vehicle detection
- Vehicle height estimation
- Speed calculation
- Curve vehicle monitoring



Fig 4.4 Ultrasonic Sensor

4.1.5 LoRa SX1278 Module (433 MHz)

The LoRa SX1278 module provides long-range wireless communication between the sensing units and the central controller.

Specifications:

- Frequency: 433 MHz
- Communication Range: Up to several kilometres (line of sight)
- Low Power Consumption
- SPI Interface

Role in the Project:

- Wireless transmission of vehicle information
- Long-range communication
- Reliable data exchange



Fig 4.5 LoRa Module

4.1.6 16×2 I²C LCD Display

The LCD displays warning messages and vehicle status to approaching drivers.

Role in the Project:

- Display road status
- Display warning messages
- Display system information



Fig 4.6 16x2 LCD Display

4.1.7 LED Indicators

LEDs provide visual warning indications regarding vehicle movement and road conditions.

Role in the Project:

- Safe to Proceed indication
- Stop indication
- Traffic status indication



Fig 4.7 LED's

4.1.8 Buzzer

The buzzer provides an audible warning whenever there is a potential collision risk.

Role in the Project:

- Audible driver alert
- Collision warning



Fig 4.8 Buzzer

4.2 SOFTWARE DESCRIPTION

The software tools used in this project are responsible for programming, compiling, debugging, and uploading the firmware to the embedded controllers.

4.2.1 Arduino IDE

Arduino IDE is the primary software used to develop and upload programs to the ESP32, ESP8266, and Arduino Nano. It provides a simple programming environment with built-in support for embedded development.

Features:

- C/C++ Programming
- Serial Monitor
- Library Management
- Code Compilation
- Firmware Upload

4.2.2 ESP32 Board Package

The ESP32 Board Package enables Arduino IDE to compile and upload programs specifically for ESP32 development boards.

Role:

- ESP32 support
- Driver installation
- Program compilation

4.2.3 ESP8266 Board Package

The ESP8266 Board Package provides support for programming the ESP8266 NodeMCU board using Arduino IDE.

Role:

- ESP8266 programming
- Library integration
- Firmware upload

4.2.4 Libraries Used

The following libraries are used in the implementation of the project:

- SPI Library – Communication with LoRa module.
- LoRa Library – Wireless data transmission and reception.
- Wire Library – I²C communication between ESP8266 and Arduino Nano.
- LiquidCrystal_I2C Library – LCD display control.

These software tools and libraries provide an efficient platform for implementing the proposed system and ensure reliable communication between the hardware modules.

CHAPTER 5

RESULTS AND DISCUSSION

The proposed Accident Prevention System at Blind Curves was successfully implemented and tested under different operating conditions to evaluate its performance. The complete prototype consists of two ESP32-based sensing units installed on opposite sides of the blind curve and a central control unit comprising an ESP8266 and an Arduino Nano. Each sensing unit was equipped with three HC-SR04 ultrasonic sensors for vehicle detection, vehicle height estimation, and speed calculation. The central control unit was connected to two ultrasonic sensors positioned at the blind curve, two 16×2 I²C LCD displays, LED indicators, and a buzzer. Communication between the sensing units and the central controller was established using LoRa SX1278 modules operating at 433 MHz.



Fig 5.1 Prototype

The system was tested by simulating vehicles approaching the blind curve from both the upper and lower directions. During each test, the ultrasonic sensors successfully detected the approaching vehicle and transmitted the collected information to the ESP32 controller. The ESP32 processed the sensor data to estimate the vehicle height, calculate the vehicle speed, and determine the estimated time required for the vehicle to reach the blind curve. The processed information was then transmitted

wirelessly to the ESP8266 through the LoRa communication module.

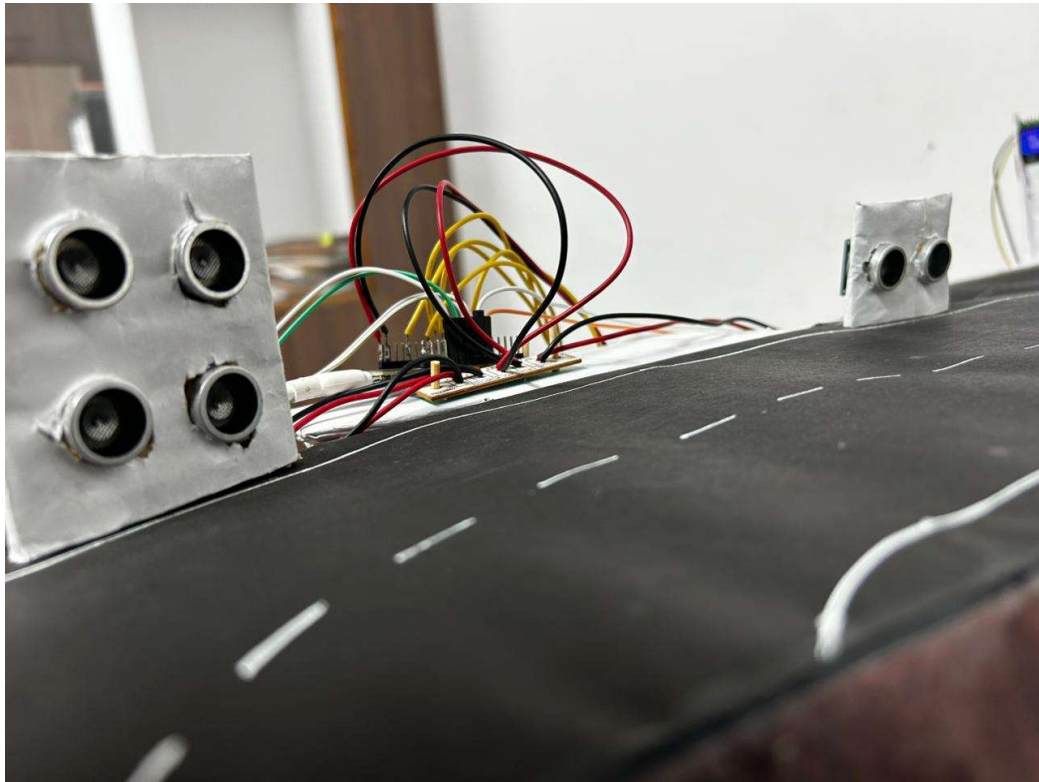


Fig 5.2 Upper Node

The ESP8266 successfully received the transmitted data from both sensing units and executed the priority decision algorithm. The controller compared the vehicle direction, estimated arrival time, speed, and vehicle type before determining the appropriate warning message. Based on the generated decision, commands were transmitted to the Arduino Nano through the I²C communication interface. The Arduino Nano controlled the LED indicators, LCD displays, and buzzer, providing clear visual and audible warnings to the drivers.

The curve-side ultrasonic sensors effectively monitored vehicle movement at the blind curve and confirmed when a vehicle had successfully crossed the dangerous section. Once the vehicle crossed the curve, the system automatically updated its status and refreshed the warning indications, allowing the next approaching vehicle to proceed safely. This continuous monitoring process ensured smooth traffic flow while reducing the possibility of head-on collisions.

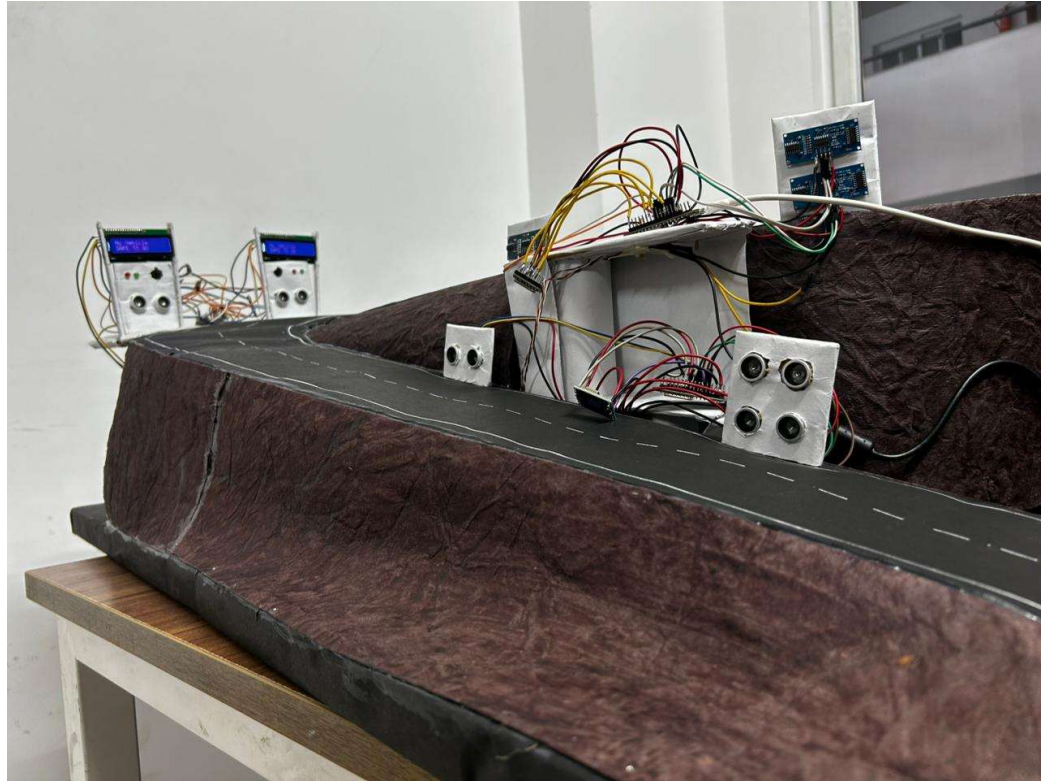


Fig 5.3 Lower Node

The LoRa communication modules provided reliable wireless communication throughout the testing process. Data packets containing vehicle direction, vehicle type, speed, and estimated arrival time were transmitted successfully with minimal delay. The wireless communication eliminated the need for long wired connections between both sides of the blind curve, making the system suitable for installation in remote and hilly regions.

The LCD displays correctly presented warning messages and traffic status information according to the decisions generated by the controller. LED indicators provided immediate visual guidance, while the buzzer produced audible warnings whenever necessary. The combined use of visual and audible indications improved driver awareness and enabled safer decision-making before entering the blind curve. The experimental results demonstrate that the proposed system is capable of continuously monitoring approaching vehicles, estimating vehicle speed and arrival time, exchanging information through LoRa communication, and generating timely

warnings based on intelligent decision-making. The modular architecture of the system allows easy installation, maintenance, and future expansion while maintaining low implementation cost.

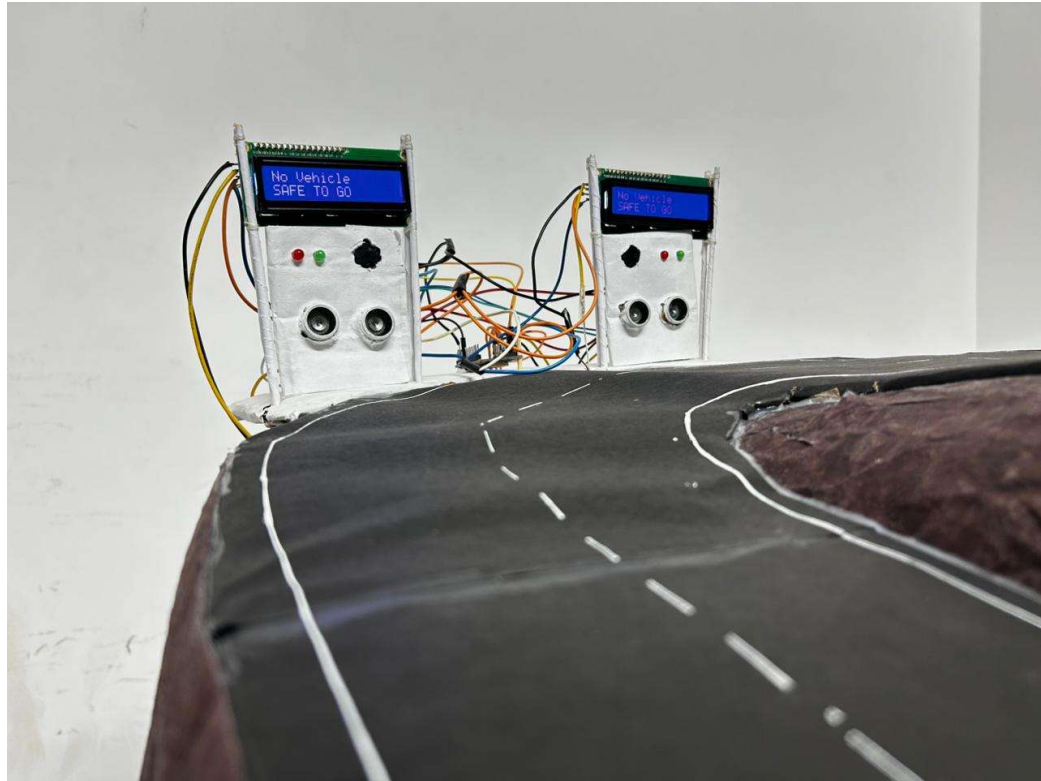


Fig 5.4 Blind Curve Node

Advantages of the Proposed System

- Real-time vehicle detection from both sides of the blind curve.
- Vehicle height estimation using ultrasonic sensors.
- Vehicle speed calculation and estimated arrival time prediction.
- Reliable long-range wireless communication using LoRa (433 MHz).
- Priority-based traffic management for safer vehicle movement.
- Automatic visual and audible warning generation using LCDs, LEDs, and buzzer.
- Low-cost implementation using readily available embedded hardware.
- Suitable for remote, hilly, and mountainous roads.

Limitations

Although the developed prototype performed satisfactorily during testing, certain limitations exist. The ultrasonic sensors have a limited sensing range and their performance may be affected by environmental conditions such as heavy rain or obstacles. The prototype was tested on a small-scale experimental setup, and additional calibration would be required before large-scale deployment. Furthermore, the current implementation does not include GPS-based vehicle tracking, weather monitoring, or cloud-based data storage, which may be considered in future enhancements.

The obtained results confirm that the proposed Accident Prevention System at Blind Curves effectively improves driver awareness by providing timely warnings and intelligent traffic management. The successful integration of ESP32, ESP8266, Arduino Nano, ultrasonic sensors, and LoRa communication demonstrates the feasibility of implementing a low-cost and reliable accident prevention system for real-world blind curve applications.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The Accident Prevention System at Blind Curves presented in this project provides an effective and economical solution for improving road safety in areas with limited visibility. The system successfully integrates embedded controllers, ultrasonic sensors, and LoRa wireless communication to continuously monitor vehicle movement on both sides of a blind curve and provide timely warnings to approaching drivers.

The developed prototype consists of two ESP32-based sensing units and a central control unit comprising an ESP8266 and an Arduino Nano. The sensing units accurately detect approaching vehicles, estimate vehicle height, calculate vehicle speed, and determine the estimated time required for the vehicle to reach the blind curve. The processed information is transmitted wirelessly using LoRa technology operating at 433 MHz, enabling reliable long-range communication with low power consumption.

The central controller processes the received information using a priority-based decision algorithm and generates appropriate warning messages through LCD displays, LED indicators, and a buzzer. The integration of curve-side ultrasonic sensors further enables the system to monitor vehicle movement across the blind curve and update the traffic status continuously. Experimental testing demonstrated that the system performs reliable vehicle detection, efficient wireless communication, and timely warning generation under different operating conditions.

The proposed system offers several advantages including low implementation cost, modular architecture, ease of installation, low power consumption, and suitability for deployment in remote and hilly regions. By providing real-time information about approaching vehicles, the system enhances driver awareness, minimizes the possibility of head-on collisions, and contributes to safer road transportation. Therefore, the proposed system can serve as a practical and reliable solution for accident prevention at blind curves.

6.2 FUTURE SCOPE

Although the proposed system achieves its primary objective of preventing accidents at blind curves, several improvements can be incorporated in future versions to further enhance its performance and functionality.

Future enhancements may include integrating GPS technology to provide the exact location of the blind curve and approaching vehicles. A GSM or Internet-based communication system can also be incorporated to transmit warning notifications to traffic authorities or connected vehicles. The system can be further improved by adding weather sensors to monitor environmental conditions such as fog, rainfall, or low visibility and generate additional safety warnings.

Artificial Intelligence and Machine Learning techniques can be integrated for more accurate vehicle classification, traffic analysis, and intelligent decision-making. Solar-powered operation may also be implemented to enable continuous operation in remote locations without dependence on external power sources. Data logging and cloud connectivity can be incorporated for real-time traffic monitoring, performance analysis, and maintenance of accident-prone locations.

The system can also be expanded to support multiple blind curves and larger road networks by increasing the number of sensing units and communication nodes. These enhancements would transform the proposed prototype into a comprehensive intelligent transportation system capable of providing improved safety, efficient traffic management, and smart road monitoring for future road infrastructure.

REFERENCES

- [1] K. V. Mutya and S. Rudra, "U-Bend Accident Prevention Through LoRa Communication," *Journal on Electronic and Automation Engineering*, vol. 4, no. 2, pp. 1–10, June 2025.
- [2] Espressif Systems, *ESP32 Series Datasheet*, Version 5.0, Espressif Systems, Shanghai, China, 2023.
- [3] Espressif Systems, *ESP8266EX Datasheet*, Version 7.0, Espressif Systems, Shanghai, China, 2020.
- [4] Arduino, *Arduino Nano Rev3 Technical Specifications*, Arduino Documentation, Arduino AG, 2024.
- [5] Semtech Corporation, *SX1276/SX1277/SX1278 LoRa Transceiver Datasheet*, Semtech Corporation, 2020.
- [6] ELECFreaks, *HC-SR04 Ultrasonic Distance Sensor User Manual and Datasheet*.
- [7] Arduino, *Arduino IDE Documentation*, Arduino AG, 2024.
- [8] Rui Santos, "ESP32 with HC-SR04 Ultrasonic Sensor using Arduino IDE," Random Nerd Tutorials, 2021.
- [9] Z. Zhang, Y. Wang, and X. Liu, "LoRaWAN-Based Vehicle-to-Vehicle Communication System for Collision Prevention," *IEEE Access*, vol. 8, pp. 124562–124574, 2020.
- [10] M. F. Ahmed, A. Khan, and S. Rahman, "Performance Assessment of LoRaWAN for Vehicular Communication," *IEEE Internet of Things Journal*, vol. 8, no. 14, pp. 11450–11461, 2021.
- [11] S. Kumar and R. Singh, "IoT-Based Smart Road Safety System Using Embedded

Technology," *International Journal of Advanced Research in Computer Science*, vol. 11, no. 3, pp. 45–52, 2020.

[12] P. Sharma and A. Verma, "Embedded System for Blind Curve Accident Prevention," *International Journal of Innovative Research in Science and Engineering*, vol. 8, no. 4, pp. 112–118, 2019.

[13] M. Banzi and M. Shiloh, *Getting Started with Arduino*, 4th ed., Maker Media, 2022.

[14] S. Monk, *Programming Arduino: Getting Started with Sketches*, 3rd ed., McGraw-Hill Education, 2021.

[15] N. Kolban, *Kolban's Book on ESP32*, Independently Published, 2020.

[16] Semtech Corporation, *LoRa Modulation Basics*, Application Note AN1200.22, Semtech Corporation, 2015.

[17] Espressif Systems, *ESP-IDF Programming Guide*, Espressif Systems, Version 5.x, 2024.

[18] Arduino, *Wire (I²C) Library Reference*, Arduino Documentation, Arduino AG, 2024.

[19] Arduino, *SPI Library Reference*, Arduino Documentation, Arduino AG, 2024.

[20] Arduino, *LiquidCrystal I²C Library Documentation*, Arduino Documentation, Arduino AG, 2024.

APPENDIX

Appendix A – Source Code

1. ESP32 Upper Sensing Unit Program

```
#include <SPI.h>

#include <LoRa.h>

#define SS    5

#define DIO0  4

// U1 - Lower Detection Sensor

#define TRIG1 14

#define ECHO1 27

// U2 - Upper Height Sensor

#define TRIG2 26

#define ECHO2 25

// U3 - Speed Sensor

#define TRIG3 33

#define ECHO3 32

#define SENSOR_DISTANCE 0.20 // Distance between U1 and U3 (20 cm)

#define CURVE_DISTANCE 200.0 // Distance from sensor to curve (meters)

bool vehiclePresent = false;

unsigned long t1 = 0;

unsigned long t2 = 0;

String vehicleType = "";

float getDistance(int trigPin, int echoPin)

{

    digitalWrite(trigPin, LOW);

    delayMicroseconds(2);

    digitalWrite(trigPin, HIGH);

    delayMicroseconds(10);

    digitalWrite(trigPin, LOW);
```

```

    long duration = pulseIn(echoPin, HIGH, 30000);
    if (duration == 0)
        return 999;
    return duration * 0.0343 / 2.0;
}

void setup()
{
    Serial.begin(115200);

    pinMode(TRIG1, OUTPUT);
    pinMode(ECHO1, INPUT);
    pinMode(TRIG2, OUTPUT);
    pinMode(ECHO2, INPUT);
    pinMode(TRIG3, OUTPUT);
    pinMode(ECHO3, INPUT);
    LoRa.setPins(SS, -1, DIO0);
    if (!LoRa.begin(433E6))
    {
        Serial.println("LoRa Initialization Failed!");
        while (1);
    }

    // Must match ESP8266 Receiver
    LoRa.setSpreadingFactor(7);
    LoRa.setSignalBandwidth(125E3);
    LoRa.setCodingRate4(5);
    LoRa.setTxPower(17);
    LoRa.setPreambleLength(8);
    LoRa.setSyncWord(0x12);
    LoRa.enableCrc();

```

```

Serial.println();
Serial.println("=====");
Serial.println("UPPER ESP32 NODE READY");
Serial.println("=====");
}

void loop()
{
    float lowerDist = getDistance(TRIG1, ECHO1);
    float upperDist = getDistance(TRIG2, ECHO2);
    float speedDist = getDistance(TRIG3, ECHO3);
    Serial.print("Lower : ");
    Serial.print(lowerDist);
    Serial.print(" cm ");
    Serial.print("Upper : ");
    Serial.print(upperDist);
    Serial.print(" cm ");
    Serial.print("Speed Sensor : ");
    Serial.print(speedDist);
    Serial.println(" cm");
    if (lowerDist < 15 && !vehiclePresent)
    {
        vehiclePresent = true;
        t1 = millis();
        if (upperDist < 15)
            vehicleType = "HEAVY";
        else
            vehicleType = "LIGHT";
        Serial.println();
        Serial.println("-----");
    }
}

```

```

Serial.println("Vehicle Detected");
Serial.print("Vehicle Type : ");
Serial.println(vehicleType);
Serial.println("-----");
}
if (vehiclePresent && speedDist < 15)
{
    t2 = millis();
    float timeSec = (t2 - t1) / 1000.0;
    if (timeSec > 0)
    {
        float speed_mps = SENSOR_DISTANCE / timeSec;
        float speed_kmph = speed_mps * 3.6;
        String packet =
            "UP," +
            vehicleType +
            "," +
            String(speed_kmph, 1) +
            "," +
            String((int)CURVE_DISTANCE);
        Serial.println();
        Serial.println("***** VEHICLE DATA *****");
        Serial.print("Type      : ");
        Serial.println(vehicleType);
        Serial.print("Time      : ");
        Serial.print(timeSec);
        Serial.println(" sec");
        Serial.print("Speed     : ");
        Serial.print(speed_kmph);

```

```

Serial.println(" km/h");
Serial.print("Distance   : ");
Serial.print(CURVE_DISTANCE);
Serial.println(" m");

Serial.print("LoRa Packet  : ");
Serial.println(packet);
LoRa.beginPacket();
LoRa.print(packet);
int status = LoRa.endPacket();
Serial.print("LoRa TX    : ");
if (status == 1)
    Serial.println("SUCCESS");
else
    Serial.println("FAILED");
Serial.println("*****");
Serial.println();
vehiclePresent = false;
delay(1000);
}
}
delay(100);
}

```

2. ESP32 Lower Sensing Unit Program

```

#include <SPI.h>
#include <LoRa.h>
#define SS    5
#define DIO0  4

```

```

// U1 - Lower Detection Sensor

#define TRIG1 14

#define ECHO1 27

// U2 - Upper Height Sensor

#define TRIG2 26

#define ECHO2 25

// U3 - Speed Sensor

#define TRIG3 33

#define ECHO3 32

#define SENSOR_DISTANCE 0.20 // Distance between U1 and U3 (20 cm)

#define CURVE_DISTANCE 200.0 // Distance from sensor to curve (meters)

bool vehiclePresent = false;

unsigned long t1 = 0;

unsigned long t2 = 0;

String vehicleType = "";

float getDistance(int trigPin, int echoPin)
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH, 30000);
    if (duration == 0)
        return 999;
    return duration * 0.0343 / 2.0;
}

void setup()
{

```

```

Serial.begin(115200);
pinMode(TRIG1, OUTPUT);
pinMode(ECHO1, INPUT);
pinMode(TRIG2, OUTPUT);
pinMode(ECHO2, INPUT);
pinMode(TRIG3, OUTPUT);
pinMode(ECHO3, INPUT);
LoRa.setPins(SS, -1, DIO0);
if (!LoRa.begin(433E6))
{
    Serial.println("LoRa Initialization Failed!");
    while (1);
}
// Must match ESP8266 Receiver
LoRa.setSpreadingFactor(7);
LoRa.setSignalBandwidth(125E3);
LoRa.setCodingRate4(5);
LoRa.setTxPower(17);
LoRa.setPreambleLength(8);
LoRa.setSyncWord(0x12);
LoRa.enableCrc();
Serial.println();
Serial.println("=====");
Serial.println("UPPER ESP32 NODE READY");
Serial.println("=====");
}
void loop()
{
    float lowerDist = getDistance(TRIG1, ECHO1);

```

```

float upperDist = getDistance(TRIG2, ECHO2);
float speedDist = getDistance(TRIG3, ECHO3);
Serial.print("Lower : ");
Serial.print(lowerDist);
Serial.print(" cm ");
Serial.print("Upper : ");
Serial.print(upperDist);
Serial.print(" cm ");
Serial.print("Speed Sensor : ");
Serial.print(speedDist);
Serial.println(" cm");
if (lowerDist < 15 && !vehiclePresent)
{
    vehiclePresent = true;
    t1 = millis();
    if (upperDist < 15)
        vehicleType = "HEAVY";
    else
        vehicleType = "LIGHT";
    Serial.println();
    Serial.println("-----");
    Serial.println("Vehicle Detected");
    Serial.print("Vehicle Type : ");
    Serial.println(vehicleType);
    Serial.println("-----");
}
if (vehiclePresent && speedDist < 15)
{
    t2 = millis();

```

```

float timeSec = (t2 - t1) / 1000.0;
if (timeSec > 0)
{
    float speed_mps = SENSOR_DISTANCE / timeSec;
    float speed_kmph = speed_mps * 3.6;
    String packet =
        "DOWN," +
        vehicleType +
        "," +
        String(speed_kmph, 1) +
        "," +
        String((int)CURVE_DISTANCE);
    Serial.println();
    Serial.println("***** VEHICLE DATA *****");
    Serial.print("Type      : ");
    Serial.println(vehicleType);
    Serial.print("Time      : ");
    Serial.print(timeSec);
    Serial.println(" sec");
    Serial.print("Speed      : ");
    Serial.print(speed_kmph);
    Serial.println(" km/h");
    Serial.print("Distance   : ");
    Serial.print(CURVE_DISTANCE);
    Serial.println(" m");
    Serial.print("LoRa Packet : ");
    Serial.println(packet);
    LoRa.beginPacket();
    LoRa.print(packet);
}

```

```

    int status = LoRa.endPacket();
    Serial.print("LoRa TX    : ");
    if (status == 1)
        Serial.println("SUCCESS");
    else
        Serial.println("FAILED");
    Serial.println("*****");
    Serial.println();
    vehiclePresent = false;
    delay(1000);
}
}
delay(100);
}

```

3. ESP8266 Central Controller Program

```

#include <SPI.h>
#include <LoRa.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
// LCD
LiquidCrystal_I2C lcdUp(0x27,16,2);
LiquidCrystal_I2C lcdDown(0x23,16,2);
// LoRa
#define LORA_SS    15
#define LORA_DIO0  16
#define UP_PASS_PIN D3
#define DOWN_PASS_PIN D4
// Nano I2C

```

```

#define NANO_ADDR 8

// System Constants

#define MAX_QUEUE 5

#define DEMO_DISTANCE 1.0

#define SCROLL_SPEED 250

#define ETA_UPDATE_TIME 1000

#define LCD_REFRESH 300

#define VEHICLE_TIMEOUT 30000

// Vehicle Structure

struct Vehicle
{
    bool active;

    char direction[6];

    char type[10];

    float speed;

    float eta;

    unsigned long receivedTime;
};

// Queue

Vehicle upQueue[MAX_QUEUE];

Vehicle downQueue[MAX_QUEUE];

int upFront=0;

int upRear=0;

int upCount=0;

int downFront=0;

int downRear=0;

int downCount=0;

// Display Messages

String upperMessage = "";

```

```

String lowerMessage = "";
byte currentPriority = 0;
// Scroll Variables
int upperScroll = 0;
int lowerScroll = 0;
unsigned long previousScroll = 0;
// Timers
unsigned long previousETA=0;
unsigned long previousLCD=0;
// Duplicate Packet Detection
String previousPacket="";
unsigned long previousPacketTime=0;
// Function Prototypes
void receiveLoRa();
void parsePacket(String packet);
void enqueueVehicle(Vehicle v);
void dequeueUP();
void dequeueDOWN();
void updateETA();
void updateDisplays();
void scrollDisplays();
void sendNanoCommand(byte cmd);
void updatePriority();
void removeExpiredVehicles();
void checkCurveSensors();
void showSafe();
void displayUpper();
void displayLower();
float calculateETA(float speed);

```

```

//SETUP

void setup()
{
  Serial.begin(115200);
  Wire.begin(D2, D1);
  pinMode(UP_PASS_PIN, INPUT_PULLUP);
  pinMode(DOWN_PASS_PIN, INPUT_PULLUP);
  lcdUp.init();
  lcdUp.backlight();
  lcdDown.init();
  lcdDown.backlight();
  lcdUp.clear();
  lcdDown.clear();
  lcdUp.setCursor(0,0);
  lcdUp.print("Blind Curve");
  lcdUp.setCursor(0,1);
  lcdUp.print("Starting...");
  lcdDown.setCursor(0,0);
  lcdDown.print("Blind Curve");
  lcdDown.setCursor(0,1);
  lcdDown.print("Starting...");
  delay(1000);
  Serial.println();
  Serial.println("=====");
  Serial.println(" Blind Curve Warning System");
  Serial.println("=====");
  Serial.println("Checking LoRa...");
  LoRa.setPins(LORA_SS,-1,LORA_DIO0);
  if(!LoRa.begin(433E6))

```

```

{
    Serial.println("FAILED");
    lcdUp.clear();
    lcdDown.clear();
    lcdUp.print("LoRa Failed");
    lcdDown.print("LoRa Failed");
    while(1);
}

LoRa.setSpreadingFactor(7);
LoRa.setSignalBandwidth(125E3);
LoRa.setCodingRate4(5);
LoRa.setTxPower(17);
LoRa.setPreambleLength(8);
LoRa.setSyncWord(0x12);
LoRa.enableCrc();
Serial.println("OK");
Serial.println("Checking LCDs...");
Serial.println("OK");
Serial.println("Checking Nano...");
Wire.beginTransmission(NANO_ADDR);
if(Wire.endTransmission() == 0)
    Serial.println("Nano Connected");
else
    Serial.println("Nano Not Found");
Serial.println();
lcdUp.clear();
lcdDown.clear();
lcdUp.print("SYSTEM READY");
lcdDown.print("SYSTEM READY");

```

```

    delay(1000);
    showSafe();
    Serial.println("=====");
    Serial.println("Receiver Ready");
    Serial.println("=====");
}

//MAIN LOOP
void loop()
{
    receiveLoRa();
    checkCurveSensors();
    if(millis()-previousETA>=ETA_UPDATE_TIME)
    {
        previousETA=millis();
        updateETA();
        removeExpiredVehicles();
        updatePriority();
        updateDisplays();
    }
    if(millis()-previousLCD>=LCD_REFRESH)
    {
        previousLCD=millis();
        updateDisplays();
    }
    scrollDisplays();
}

// RECEIVE LORA PACKET
void receiveLoRa()
{

```

```

int packetSize = LoRa.parsePacket();
if (!packetSize)
    return;
String packet = "";
while (LoRa.available())
{
    packet += (char)LoRa.read();
}
Serial.println();
Serial.println("===== LoRa Packet =====");
Serial.println(packet);
Serial.print("RSSI : ");
Serial.println(LoRa.packetRssi());
// Reject duplicate packet within 2 seconds
if (packet == previousPacket &&
    millis() - previousPacketTime < 500)
{
    Serial.println("Duplicate Packet Ignored");
    return;
}
previousPacket = packet;
previousPacketTime = millis();
parsePacket(packet);
updatePriority();
updateDisplays();
printQueues();
}
//PARSE PACKET
void parsePacket(String packet)

```

```

{
    Vehicle v;
    int p1 = packet.indexOf(',');
    int p2 = packet.indexOf(',', p1 + 1);
    int p3 = packet.indexOf(',', p2 + 1);
    if (p1 < 0 || p2 < 0 || p3 < 0)
    {
        Serial.println("Invalid Packet");
        return;
    }
    String dir = packet.substring(0, p1);
    String type = packet.substring(p1 + 1, p2);
    float speed = packet.substring(p2 + 1, p3).toFloat();
    dir.toCharArray(v.direction, sizeof(v.direction));
    type.toCharArray(v.type, sizeof(v.type));
    v.speed = speed;
    v.eta = calculateETA(speed);
    v.receivedTime = millis();
    v.active = true;
    enqueueVehicle(v);
    printQueues();
    Serial.println("Vehicle Added To Queue");
    Serial.print("Direction : ");
    Serial.println(v.direction);
    Serial.print("Type : ");
    Serial.println(v.type);
    Serial.print("Speed : ");
    Serial.println(v.speed);
    Serial.print("ETA : ");

```

```

    Serial.println(v.eta);
}
//ENQUEUE
void enqueueVehicle(Vehicle v)
{
    if (strcmp(v.direction, "UP") == 0)
    {
        if (upCount >= MAX_QUEUE)
        {
            Serial.println("UP Queue Full");
            Serial.println("Removing Oldest Vehicle");
            dequeueUP();
        }
        upQueue[upRear] = v;
        upRear++;
        if (upRear >= MAX_QUEUE)
            upRear = 0;
        upCount++;
        return;
    }
    if (strcmp(v.direction, "DOWN") == 0)
    {
        if (downCount >= MAX_QUEUE)
        {
            Serial.println("DOWN Queue Full");
            Serial.println("Removing Oldest Vehicle");
            dequeueDOWN();
        }
        downQueue[downRear] = v;
    }
}

```

```

    downRear++;
    if (downRear >= MAX_QUEUE)
        downRear = 0;
    downCount++;
    return;
}
}

//ETA
float calculateETA(float speed)
{
    if (speed <= 0.1)
        return 999;
    return DEMO_DISTANCE / (speed / 3.6);
}

//UPDATE ETA
void updateETA()
{
    for(int i=0;i<upCount;i++)
    {
        int idx=(upFront+i)%MAX_QUEUE;

        if(upQueue[idx].active)
        {
            upQueue[idx].eta -= 1.0;
            if(upQueue[idx].eta<0)
                upQueue[idx].eta = 0;
        }
    }
    for(int i=0;i<downCount;i++)

```

```

{
    int idx=(downFront+i)%MAX_QUEUE;
    if(downQueue[idx].active)
    {
        downQueue[idx].eta -= 1.0;
        if(downQueue[idx].eta<0)
            downQueue[idx].eta = 0;
    }
}

//REMOVE EXPIRED
void removeExpiredVehicles()
{
    // Do nothing.

    // Vehicles are removed only when the Nano
    // detects them at the curve.
}

//SAFE DISPLAY
void showSafe()
{
    lcdUp.clear();
    lcdUp.setCursor(0,0);
    lcdUp.print("No Vehicle");
    lcdUp.setCursor(0,1);
    lcdUp.print("SAFE TO GO");
    lcdDown.clear();
    lcdDown.setCursor(0,0);
    lcdDown.print("No Vehicle");
    lcdDown.setCursor(0,1);

```

```

    lcdDown.print("SAFE TO GO");
}
//LCD UPDATE
void updateDisplays()
{
    displayUpper();
    displayLower();
}
// SCROLL
void scrollDisplays()
{
    if(millis() - previousScroll < SCROLL_SPEED)
        return;
    previousScroll=millis();
    upperScroll++;
    lowerScroll++;
    if(upperScroll>=upperMessage.length())
        upperScroll=0;
    if(lowerScroll>=lowerMessage.length())
        lowerScroll=0;
}
//SEND COMMAND TO NANO
void sendNanoCommand(byte cmd)
{
    Wire.beginTransmission(NANO_ADDR);
    Wire.write(cmd);
    Wire.endTransmission();
}
// UPDATE PRIORITY

```

```

void updatePriority()
{
    if(upCount==0 && downCount==0)
    {
        currentPriority = 0;
        sendNanoCommand(currentPriority);
        showSafe();
        Serial.println("STATE : SAFE");
        return;
    }
    if(upCount>0 && downCount==0)
    {
        currentPriority = 1;
        sendNanoCommand(currentPriority);
        Serial.println("STATE : ABOVE ONLY");
        return;
    }
    if(upCount==0 && downCount>0)
    {
        currentPriority = 2;
        sendNanoCommand(currentPriority);
        Serial.println("STATE : BELOW ONLY");
        return;
    }
    bool upHeavy =
        strcmp(upQueue[upFront].type,"HEAVY")==0;
    bool downHeavy =
        strcmp(downQueue[downFront].type,"HEAVY")==0;
    if(upHeavy && !downHeavy)

```

```

{
    currentPriority = 1;
    sendNanoCommand(currentPriority);
    Serial.println("STATE : ABOVE HEAVY");
    return;
}
if(!upHeavy && downHeavy)
{
    currentPriority = 2;
    sendNanoCommand(currentPriority);
    Serial.println("STATE : BELOW HEAVY");
    return;
}
// Heavy vs Heavy OR Light vs Light
currentPriority = 2;

sendNanoCommand(currentPriority);

Serial.println("STATE : BELOW PRIORITY");
}

//DISPLAY UPPER
void displayUpper()
{
    lcdUp.clear();
    if(downCount==0)
    {
        lcdUp.setCursor(0,0);
        lcdUp.print("No Vehicle");
    }
}

```

```

        lcdUp.setCursor(0,1);
        lcdUp.print("SAFE TO GO");
        upperMessage="";
        return;
    }
    if(strcmp(downQueue[downFront].type,"HEAVY")==0)
        upperMessage="Heavy Vehicle From Below";
    else
        upperMessage="Light Vehicle From Below";
    if(upperScroll>=upperMessage.length())
        upperScroll=0;
    lcdUp.setCursor(0,0);
    String temp = upperMessage + "   ";
    lcdUp.print(temp.substring(upperScroll, upperScroll + 16));
    if(downQueue[downFront].eta>0)
    {
        lcdUp.setCursor(0,1);
        lcdUp.print("ETA:");
        if((int)downQueue[downFront].eta<10)
            lcdUp.print("0");
        lcdUp.print((int)downQueue[downFront].eta);
        lcdUp.print(" ");
        if(currentPriority == 1)
            lcdUp.print("PROCEED");
        else
            lcdUp.print("GO SLOW");
    }
    else
    {

```

```

        lcdUp.setCursor(0,1);
        if(currentPriority == 1)
            lcdUp.print("PROCEED");
        else
            lcdUp.print("GO SLOW");
    }
}

//          DISPLAY LOWER
void displayLower()
{
    lcdDown.clear();

    if(upCount==0)
    {
        lcdDown.setCursor(0,0);
        lcdDown.print("No Vehicle");
        lcdDown.setCursor(0,1);
        lcdDown.print("SAFE TO GO");
        lowerMessage="";
        return;
    }
    if(strcmp(upQueue[upFront].type,"HEAVY")==0)
        lowerMessage="Heavy Vehicle From Above";
    else
        lowerMessage="Light Vehicle From Above";
    if(lowerScroll>=lowerMessage.length())
        lowerScroll=0;
    lcdDown.setCursor(0,0);
    String temp = lowerMessage + "  ";

```

```

lcdDown.print(temp.substring(lowerScroll, lowerScroll + 16));
if(upQueue[upFront].eta>0)
{
    lcdDown.setCursor(0,1);

    lcdDown.print("ETA:");
    if((int)upQueue[upFront].eta<10)
        lcdDown.print("0");
    lcdDown.print((int)upQueue[upFront].eta);
    lcdDown.print(" ");
    if(currentPriority == 2)
        lcdDown.print("PROCEED");
    else
        lcdDown.print("GO SLOW");
}
else
{
    lcdDown.setCursor(0,1);
    if(currentPriority == 2)
        lcdDown.print("PROCEED");
    else
        lcdDown.print("GO SLOW");
}
}

//SERIAL DEBUG
void printQueues()
{
    Serial.println();
    Serial.println("===== QUEUE =====");
}

```

```

Serial.print("UP Queue  : ");
Serial.println(upCount);
for(int i=0;i<upCount;i++)
{
    int idx=(upFront+i)%MAX_QUEUE;
    Serial.print(i+1);
    Serial.print(" -> ");
    Serial.print(upQueue[idx].type);
    Serial.print(" ");
    Serial.print(upQueue[idx].speed);
    Serial.print(" km/h  ETA ");
    Serial.println(upQueue[idx].eta);
}

Serial.println();
Serial.print("DOWN Queue : ");
Serial.println(downCount);
for(int i=0;i<downCount;i++)
{
    int idx=(downFront+i)%MAX_QUEUE;
    Serial.print(i+1);
    Serial.print(" -> ");
    Serial.print(downQueue[idx].type);
    Serial.print(" ");
    Serial.print(downQueue[idx].speed);
    Serial.print(" km/h  ETA ");
    Serial.println(downQueue[idx].eta);
}
Serial.println("=====");

```

```

}

void checkCurveSensors()
{
    if(digitalRead(UP_PASS_PIN)==LOW)
    {
        Serial.println("UP Vehicle Passed");
        dequeueUP();
        printQueues();
        updatePriority();
        updateDisplays();
        while(digitalRead(UP_PASS_PIN)==LOW);
    }
    if(digitalRead(DOWN_PASS_PIN)==LOW)
    {
        Serial.println("DOWN Vehicle Passed");
        dequeueDOWN();
        printQueues();
        updatePriority();
        updateDisplays();
        while(digitalRead(DOWN_PASS_PIN)==LOW);
    }
}

//DEQUEUE UP
void dequeueUP()
{
    if (upCount == 0)
        return;
    upQueue[upFront].active = false;
    upFront++;
}

```

```

    if (upFront >= MAX_QUEUE)
        upFront = 0;
    upCount--;
    Serial.println("UP Queue Updated");
    Serial.print("Remaining : ");
    Serial.println(upCount);
}

DEQUEUE DOWN
void dequeueDOWN()
{
    if (downCount == 0)
        return;

    downQueue[downFront].active = false;
    downFront++;
    if (downFront >= MAX_QUEUE)
        downFront = 0;
    downCount--;
    Serial.println("DOWN Queue Updated");
    Serial.print("Remaining : ");
    Serial.println(downCount);
}

```

4. Arduino Nano Output Controller Program

```

#include <Wire.h>

#define I2C_ADDRESS 8

// Upper Curve Ultrasonic
#define TRIG_UP 2
#define ECHO_UP 3

// Lower Curve Ultrasonic

```

```

#define TRIG_DOWN 4
#define ECHO_DOWN 5
// LEDs
#define UP_GREEN 6
#define UP_RED 7
#define DOWN_GREEN 8
#define DOWN_RED 9
// Buzzer
#define BUZZER 10
// ESP8266 Signal Pins
#define UP_PASS 11
#define DOWN_PASS 12
// Variables
byte currentState = 0;
bool upperDetected = false;
bool lowerDetected = false;
unsigned long buzzerTimer = 0;
bool buzzerState = false;
// Function Prototypes
float getDistance(int trigPin,int echoPin);
void updateOutputs();
void receiveEvent(int bytes);
void checkCurveSensors();
void pulseUP();
void pulseDOWN();
//SETUP
void setup()
{
    Serial.begin(115200);

```

```

pinMode(TRIG_UP,OUTPUT);
pinMode(ECHO_UP,INPUT);
pinMode(TRIG_DOWN,OUTPUT);
pinMode(ECHO_DOWN,INPUT);
pinMode(UP_GREEN,OUTPUT);
pinMode(UP_RED,OUTPUT);
pinMode(DOWN_GREEN,OUTPUT);
pinMode(DOWN_RED,OUTPUT);
pinMode(BUZZER,OUTPUT);
pinMode(UP_PASS,OUTPUT);
pinMode(DOWN_PASS,OUTPUT);
digitalWrite(UP_PASS,HIGH);
digitalWrite(DOWN_PASS,HIGH);
Wire.begin(I2C_ADDRESS);
Wire.onReceive(receiveEvent);
Serial.println();
Serial.println("=====");
Serial.println("Nano Controller Ready");
Serial.println("=====");
updateOutputs();
}

//LOOP

void loop()
{
    checkCurveSensors();
    updateOutputs();
}

//RECEIVE I2C COMMAND

void receiveEvent(int howMany)

```

```

{
  if(Wire.available())
  {
    currentState = Wire.read();
    Serial.print("Received State : ");
    Serial.println(currentState);
  }
}

UPDATE OUTPUTS

void updateOutputs()
{
  switch(currentState)
  {
    // SAFE
    case 0:
      digitalWrite(UP_GREEN,HIGH);
      digitalWrite(UP_RED,LOW);
      digitalWrite(DOWN_GREEN,HIGH);
      digitalWrite(DOWN_RED,LOW);
      digitalWrite(BUZZER,LOW);
      break;
    // ABOVE VEHICLE ONLY
    case 1:
      digitalWrite(UP_GREEN,HIGH);
      digitalWrite(UP_RED,LOW);
      digitalWrite(DOWN_GREEN,LOW);
      digitalWrite(DOWN_RED,HIGH);
      digitalWrite(BUZZER,LOW);
      break;
  }
}

```

```

// BELOW VEHICLE ONLY

case 2:
    digitalWrite(UP_GREEN,LOW);
    digitalWrite(UP_RED,HIGH);
    digitalWrite(DOWN_GREEN,HIGH);
    digitalWrite(DOWN_RED,LOW);
    digitalWrite(BUZZER,LOW);
    break;

// BOTH SIDES

case 3:
    digitalWrite(UP_GREEN,LOW);
    digitalWrite(UP_RED,HIGH);
    digitalWrite(DOWN_GREEN,HIGH);
    digitalWrite(DOWN_RED,LOW);
    if(millis()-buzzerTimer>=500)
    {
        buzzerTimer=millis();
        buzzerState=!buzzerState;
        digitalWrite(BUZZER,buzzerState);
    }
    break;
}

}

//DISTANCE

float getDistance(int trigPin,int echoPin)
{
    digitalWrite(trigPin,LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin,HIGH);

```

```

    delayMicroseconds(10);
    digitalWrite(trigPin,LOW);
    long duration=pulseIn(echoPin,HIGH,30000);
    if(duration==0)
        return 999;
    return duration*0.0343/2.0;
}

//      CHECK CURVE SENSORS
void checkCurveSensors()
{
    float upDistance=getDistance(TRIG_UP,ECHO_UP);
    float downDistance=getDistance(TRIG_DOWN,ECHO_DOWN);
    if(upDistance<15 && !upperDetected)
    {
        upperDetected=true;
        Serial.println("Upper Vehicle Passed");
        pulseUP();
    }
    if(upDistance>20)
        upperDetected=false;
    if(downDistance<15 && !lowerDetected)
    {
        lowerDetected=true;
        Serial.println("Lower Vehicle Passed");
        pulseDOWN();
    }
    if(downDistance>20)
        lowerDetected=false;
}

```

```
//PULSE TO ESP8266

void pulseUP()
{
    digitalWrite(UP_PASS,LOW);
    delay(100);
    digitalWrite(UP_PASS,HIGH);
}

//          PULSE TO ESP8266

void pulseDOWN()
{
    digitalWrite(DOWN_PASS,LOW);
    delay(100);
    digitalWrite(DOWN_PASS,HIGH);
}
```

Appendix B – Hardware Components

S.No	Component	Specification	Quantity	Cost (₹)
1	ESP32 Development Board	ESP32 DevKit V1	2	800
2	ESP8266 NodeMCU	ESP8266 ESP-12E	1	200
3	Arduino Nano	ATmega328P	1	210
4	LoRa SX1278 Module	433 MHz	3	840
5	HC-SR04 Ultrasonic Sensor	Distance Sensor	8	680
6	16×2 LCD Display with I ² C Module	Display Unit	2	360
7	LED Indicators	5 mm LEDs	4	15
8	Active Buzzer	5 V	1	10
9	Jumper Wires	Male-Male / Male-Female	As Required	50
10	Power Supply	5 V DC Adapter	3	30
11	Miscellaneous	-	-	~300

Total Cost (₹): 3500 (Approx.)